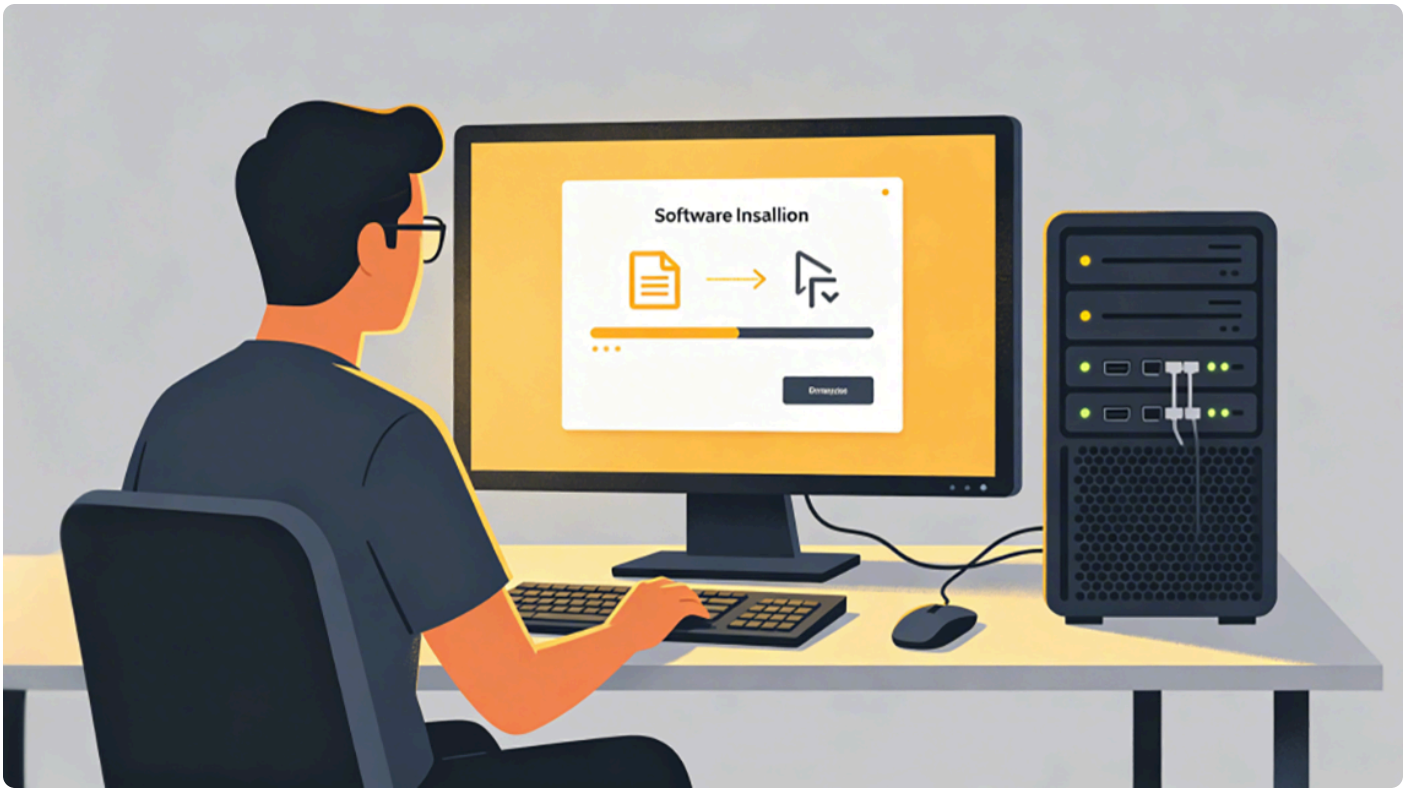
28｜容器化与部署：从本地到可交付

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

Hify 的所有功能开发完了，测试也补了。但它现在只能在你的开发机上跑，换一台机器就要装 JDK、装 Node.js、配环境变量，折腾半天。

这一讲把它变成可交付的。和之前学 RAG 一样，你不需要懂 Dockerfile 和 K8s YAML 的语法细节，用自然语言告诉 Claude Code 你要什么，它帮你生成，你执行命令就行。

先和 Claude Code 讨论部署形态

不急着生成任何文件。先用咨询模式和 Claude Code 讨论清楚要支持哪些部署形态，有什么需要提前准备的。

```
1 Hify 是一个 Spring Boot + Vue 的 AI 应用，
2 后端调用外部 LLM API，依赖 MySQL、Redis、pgvector，
```

复制代码

```
目标用户是企业内部团队，规模从几人到几十人不等。

我想把它做成可交付的形态，方便部署到不同环境。
帮我分析应该支持哪些部署形态，以及每种形态需要提前准备什么。
不要写任何代码，先给我分析
```

Claude Code 的分析给出了三种部署形态，以及每种形态需要准备的内容。

形态一：tar.gz 本地部署

适合客户内网无 Docker、纯 Linux 环境，运维能力弱的场景。

需要准备：fat jar + 前端 dist、application.yml 配置模板、start.sh / stop.sh / status.sh、Nginx 配置模板（含 SSE 配置）、数据库初始化 schema.sql。目标机器需要自行安装 JRE 17+、MySQL、Redis、PostgreSQL + pgvector。

形态二：Docker 部署

适合有 Docker 环境的技术型中小团队。

需要准备：前后端各自的 Dockerfile（多阶段构建）、docker-compose.yml、.env 模板。pgvector 有官方镜像 `pgvector/pgvector:pg16`，比本地部署友好很多。

形态三：K8s 部署

适合有 K8s 集群、追求高可用的企业。

需要准备：Deployment、Service、ConfigMap、Secret、Ingress 等 YAML 文件。MySQL/Redis 通常复用客户已有实例，pgvector 需要单独处理。

Claude Code 还给出了三种形态的横向对比和建议优先级：**先做 Docker（投入产出比最高）、再做 tar.gz（面向传统企业）、最后做 K8s（复用 Docker 镜像）**。这里我们按照从简单到复杂的顺序来做。

Claude Code 主动提出的四个确认问题：

1. MySQL、Redis、pgvector 是跟 Hify 一起部署，还是连接已有的外部服务？外部服务，通过配置连接
2. 目标机器有没有 Java 环境？有，不需要打包 JDK
3. Docker 部署是否把数据库也包进去？不包含，只部署 Hify 自身
4. 当前 Hify 有没有 HTTP 健康检查接口？三种部署形态都需要它，本地部署用来确认服务起来了，Docker 用来做 healthcheck 指令，K8s 用来做存活和就绪探针。如果没有，需要先补。

前三个问题确定了部署边界，第四个是我没想到的遗漏，Claude Code 主动提醒了一个前置工作。这是咨询模式的价值：不是让 AI 直接执行，而是先讨论清楚，让它帮你发现遗漏，再动手。

先补健康检查接口

Claude Code 提醒了，先把前置工作做完，再做部署。

复制代码

```
帮 Hify 加一个健康检查接口 GET /api/v1/health，
返回 HTTP 200 和 {"code":0,"data":"ok"}。

这个接口会被三个地方用到：
- 本地部署：启动后 curl 确认服务起来了
- Docker：healthcheck 指令
- K8s：liveness 和 readiness 探针

接口本身只需要返回 ok，不需要检查数据库连接。
```

Claude Code 没有直接生成代码，而是先扫描了项目，发现 `HealthController.java` 已经存在了，接口也有。但它主动发现了一个格式不一致的问题：

现有接口返回的是 `{"code":200,"message":"success","data":"ok"}`，和要求的 `{"code":0,"data":"ok"}` 有差异，code 值不同，多了 message 字段。

Claude Code 给出了两个选项：

1. 保持现状：健康检查场景里 curl/Docker/K8s 只看 HTTP 状态码，不解析 body，code 值无所谓。
2. 改成 code:0：需要改动，但会影响所有接口的响应格式。

Claude Code 的建议是，选 1，不必为健康检查改全局响应格式。

确认选 1 之后，Claude Code 直接修改了 `HealthController.java`，在这个接口单独返回 `Map.of("code", 0, "data", "ok")`，不动全局 `Result` 类，改动最小。

顺带给出了三种部署形态下怎么使用健康检查接口的方式：

1. tag.gz 形态

```
curl -f http://localhost:8080/api/v1/health
```

2. Docker 形态

```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
  CMD curl -f http://localhost:8080/api/v1/health || exit 1
```

3. K8s 形态：

```
# K8s probe
```

```
livenessProbe:
  httpGet:
    path: /api/v1/health
    port: 8080
  initialDelaySeconds: 30
  periodSeconds: 10
readinessProbe:
  httpGet:
    path: /api/v1/health
    port: 8080
  initialDelaySeconds: 10
  periodSeconds: 5
```

这个交互有两个值得注意的点：第一，Claude Code 没有盲目执行，而是先查了项目里有没有，避免重复创建；第二，发现格式不一致后主动问了你，没有自作主张改全局格式。这是咨询模式和执行模式结合的正确节奏。

本地部署：打成 tar 包

本地部署的目标：一个 tar 包，解压、填配置、执行 start.sh，服务跑起来。目标机器只需要有 Java 环境。

复制代码

```
帮 Hify 生成本地部署的打包脚本。

要求：
- 产物是一个 tar 包，包含：后端 jar、前端 dist 目录、start.sh、stop.sh、配置模板 applicat
- 目标机器已有 Java 环境，不需要打包 JDK
- start.sh 支持通过环境变量或配置文件注入 MySQL、Redis、pgvector 的连接信息
- stop.sh 优雅停止，等待进程退出
- Makefile 加 package 命令，一键打包

不需要包含 MySQL、Redis、pgvector，它们是外部服务。
```

Claude Code 没有直接生成脚本，而是先读了现有的 start.sh、stop.sh、Makefile 和 application.yml，发现：现有 start.sh 是开发模式，跑的是 `mvn build` + `npm dev server`，不适合生产部署，需要完全重写。

确认情况后，CC 生成了五个文件：

1. start.sh（完全重写）

生产模式直接运行 jar，不再跑 mvn/npm。关键逻辑：

- 启动前检查 Java 版本是否 17+，jar 文件是否存在
- 自动加载同目录的 `.env` 文件，再接受环境变量覆盖
- 外部 `application.yml` 优先级高于 jar 内置配置
- 启动后轮询 `/api/v1/health` 确认就绪，最多等 90s；进程异常退出会立即打印最后 30 行日志

2. stop.sh（精简）

去掉了前端进程管理，只管后端。发 SIGTERM，等最多 30s，超时强制 SIGKILL。

3. deploy/application.yml.template

打包时复制为 application.yml，所有连接信息用 `${ENV_VAR:default}` 占位符，用户只需要填实际的地址和密码。

4. deploy/env.template

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_NAME=hify
DB_USERNAME=hify
DB_PASSWORD=your_mysql_password

REDIS_HOST=127.0.0.1
REDIS_PORT=6379
REDIS_PASSWORD=

JVM_OPTS=-Xms256m -Xmx512m
```

用户解压后 `cp env.template .env`，填好密码就可以启动。

5. Makefile 的 package 命令

`make package` 先触发 `build`（编译后端 + 打包前端），再把产物打成 tar 包：

```
hify-0.0.1-SNAPSHOT/
├── hify-app.jar        # 后端 fat jar
├── web/                # 前端静态文件，给 Nginx 托管
├── start.sh
├── stop.sh
├── application.yml     # 配置模板
└── env.template        # 环境变量模板
```

目标机器的部署流程：

```
tar -xzf hify-0.0.1-SNAPSHOT.tar.gz
cd hify-0.0.1-SNAPSHOT
cp env.template .env && vi .env    # 填写 DB/Redis 密码
bash start.sh
```

start.sh 启动后会自动轮询健康检查接口，就绪了打印“Hify 启动成功”，失败了打印日志告诉你哪里出了问题。

Docker 部署：Dockerfile + docker-compose

Docker 部署的目标：两个 Dockerfile 分别打包前后端镜像，一条命令启动。

后端 Dockerfile

复制代码

```
帮我写 Hify 后端的 Dockerfile。

情况说明：
- Maven 多模块项目，主模块是 hify-app
- JDK 17
- 不需要打包 MySQL、Redis、pgvector，它们是外部服务

要求：
- 多阶段构建，减小镜像体积
- 非 root 用户运行（安全最佳实践）
- 健康检查用 /api/v1/health 接口
```

Claude Code 生成的后端 Dockerfile 有几个值得关注的细节：

1. **层缓存优化**：先单独 Copy 所有 pom 文件执行 `dependency:go-offline`，再 Copy 源码编译。依赖没变时直接命中缓存，省去 3-5 分钟下载时间。运行阶段还用了 Spring Boot layertools 把 jar 拆成四层（dependencies → loader → snapshot-dependencies → application），每次只改业务代码时只有最后一层失效。
2. **安全**：构建用 JDK alpine，运行只用 JRE alpine，镜像体积差约 200MB。`addgroup/adduser` 创建无 home 目录、无 shell 的系统用户，进程以非 root 运行。加了 `-Djava.security.egd=file:/dev/./urandom`，避免容器内熵不足导致启动慢。
3. **健康检查**：用 `wget`（alpine 内置）而非 curl（需额外安装），减少镜像层。`--start-period=60s` 给 Spring Boot 足够的初始化时间，这段时间内失败不计入重试。

4. **外部配置注入**：挂载 `/app/config/application.yml` 覆盖内置配置，用 `optional:` 前缀保证没挂载时也能启动。

前端 Dockerfile

复制代码

```
帮我写 Hify 前端的 Dockerfile。

情况说明：
- Vue 3 项目，npm run build 打包
- 用 Nginx 托管静态文件
- 前端需要把 /api 请求反向代理到后端

特别注意：
- Hify 有流式响应（SSE），Nginx 需要关闭缓冲
- LLM 调用可能很慢，超时时间要够长
```

Claude Code 先读了 `package.json` 和 `vite.config.ts` 确认项目结构，然后生成了两个文件。

前端 Dockerfile 同样两阶段：Node 镜像打包出 dist，只把 dist 复制进 Nginx 镜像，最终运行镜像约 40MB，不含 Node.js。

Nginx 配置是关键，Claude Code 给出了几个针对 Hify 的特殊处理：

复制代码

```
location /api/ {
    proxy_pass          http://backend:8080;

    # LLM 调用可能很慢，给足超时时间
    proxy_read_timeout    120s;

    # SSE 必须关闭缓冲，否则事件被攒批，前端收不到实时流
    proxy_buffering       off;
    proxy_cache           off;

    # SSE 长连接保活
    proxy_set_header      Connection "";
}
```

`proxy_buffering off` 是 SSE 的关键，如果不关，Nginx 会把事件攒批后一次性发出，用户看不到打字机效果，要等 LLM 全部输出完才能看到内容。`proxy_read_timeout 120s` 比后端超时稍长，避免 Nginx 先断连。

`proxy_pass http://backend:8080` 用的是 Docker 网络服务名，和后面 docker-compose 里的服务名直接对应。

生成 docker-compose.yml

复制代码

```
帮我生成 Hify 的 docker-compose.yml。

要求：
- 包含前端和后端两个服务
- MySQL、Redis、pgvector 是外部服务，通过环境变量配置连接地址
- 后端健康检查用 /api/v1/health 接口
- 前端依赖后端健康检查通过后才启动
- 敏感配置（密码、API Key）从 .env 文件读取攒批
```

Claude Code 先读了 env.template 确认变量名，再生成 docker-compose.yml，同步更新了 .env 模板加入 Docker 特有的端口映射变量。

几个值得注意的设计：

1. `depends_on: condition: service_healthy`：前端容器等后端健康检查通过后才启动，避免 Nginx 上线时后端还没就绪就开始接流量，出现 502。
2. 敏感配置隔离：docker-compose.yml 里只写变量名，所有密码和地址从 .env 读取，.env 不进 git。
3. 外部服务不能用 localhost：MySQL/Redis/pgvector 完全不在 compose 里定义，只传连接地址。Claude Code 在注释里特别提醒：目标机器上的外部服务要用真实 IP，不能用 localhost，容器内的 localhost 是容器自身，不是宿主机。
4. 端口映射可配置：宿主机 80/8080 被占时，只改 .env 里的 `FRONTEND_EXPOSE_PORT` / `BACKEND_EXPOSE_PORT`，不需要动 compose 文件。

启动方式：

复制代码

```
cp deploy/env.template .env
vi .env                          # 填写真实 IP 和密码
docker compose up -d --build
docker compose ps                # 确认两个服务都 Running
docker compose logs -f backend # 看后端启动日志
```

K8s 部署

K8s 部署的目标：前后端各自的 Deployment 和 Service，加上 Secret 和 ConfigMap 管理配置，共六个文件。

后端 Deployment 和 Service

复制代码

```
帮我写 Hify 后端的 K8s 部署文件，包含 Deployment 和 Service。

要求：
- 初始 2 个副本，支持水平扩容
- 配置和密码从 ConfigMap 和 Secret 读取，不硬编码在镜像里
- liveness 和 readiness 探针都用 /api/v1/health 接口
```

```
- 设置资源限制，防止单个 Pod 占用过多资源
- Service 类型 ClusterIP，不对外暴露，只在集群内访问
```

前端 Deployment 和 Service

复制代码

```
帮我写 Hify 前端的 K8s 部署文件，包含 Deployment 和 Service。

要求：
- 初始 2 个副本
- 资源需求比后端小（Nginx 托管静态文件）
- Service 类型 NodePort，对外暴露访问
```

Claude Code 生成前端文件前先读了已有的后端 Service，确认命名风格和 namespace 保持一致。

前后端资源配置对比：

	后端	前端
requests cpu	250m	50m
requests memory	512Mi	64Mi
limits cpu	1000m	200m
limits memory	1Gi	128Mi

Nginx 只做静态文件托管 + 反向代理，不跑 JVM，资源需求低一个数量级。前端探针的 `initialDelaySeconds` 也比后端短很多，Nginx 秒级启动，不需要等 60s。

探针路径用 `/api/v1/health` 而不是 `/`：前端健康检查通过 Nginx 代理探后端接口，同时验证了静态托管和 API 代理两条路径都通，比只探 `/` 更有价值。

前端 Service 用 `NodePort: 30080`，访问方式是 `http://<任意节点IP>:30080`。端口冲突时只改这一个字段。

Secret 和 ConfigMap

```
帮我生成 K8s 的 Secret 和 ConfigMap 模板。

Secret 存储：数据库密码、Redis 密码、pgvector 密码、OpenAI API Key
ConfigMap 存储：数据库地址端口、Redis 地址端口、pgvector 地址端口
```

Claude Code 先读了已有文件，发现 Secret 里缺少 `OPENAI_API_KEY`，直接在原文件上更新，同时修改了 backend-deployment.yml 把新增的 key 注入进去。

ConfigMap 填好地址后直接 apply：

```
vi deploy/k8s/backend-configmap.yml   # 填写真实 host
kubectl apply -f deploy/k8s/backend-configmap.yml
```

Secret 通过 CI/CD 环境变量注入，不把密码写进文件：

```
kubectl create secret generic hify-backend-secret \
  --from-literal=DB_PASSWORD=$DB_PASSWORD \
  --from-literal=REDIS_PASSWORD=$REDIS_PASSWORD \
  --from-literal=PGVECTOR_PASSWORD=$PGVECTOR_PASSWORD \
  --from-literal=OPENAI_API_KEY=$OPENAI_API_KEY \
  -n hify --dry-run=client -o yaml | kubectl apply -f -
```

`--dry-run=client -o yaml | kubectl apply -f -` 这个写法支持幂等更新，Secret 已存在时会覆盖而不报错，适合 CI/CD 每次部署时执行。

所有文件放在 `deploy/k8s/` 下：

```
deploy/k8s/
  ├── backend-deployment.yml
  ├── backend-service.yml
  ├── frontend-deployment.yml
  ├── frontend-service.yml
  ├── backend-configmap.yml
  └── backend-secret.yml
```

执行与验收

三种形态的执行命令。

本地部署：

```
make package                                  # 打包成 tar.gz
scp hify-*.tar.gz user@target-server:~        # 传到目标机器
ssh user@target-server
tar -xzf hify-*.tar.gz && cd hify-*
cp env.template .env && vi .env               # 填写 DB/Redis 密码
bash start.sh                                 # 启动，自动轮询健康检查
```

Docker 部署：

```
docker compose build                          # 构建镜像
cp deploy/env.template .env && vi .env        # 填写外部服务地址和密码
docker compose up -d                          # 启动
docker compose ps                             # 确认两个服务都 Running
docker compose logs -f backend                # 看启动日志
```

K8s 部署：

复制代码

```
kubectl create namespace hify
vi deploy/k8s/backend-configmap.yml                # 填写外部服务地址
kubectl apply -f deploy/k8s/backend-configmap.yml

# Secret 通过环境变量注入，不写进文件
kubectl create secret generic hify-backend-secret \
  --from-literal=DB_PASSWORD=$DB_PASSWORD \
  --from-literal=PGVECTOR_PASSWORD=$PGVECTOR_PASSWORD \
  --from-literal=OPENAI_API_KEY=$OPENAI_API_KEY \
  -n hify --dry-run=client -o yaml | kubectl apply -f -

kubectl apply -f deploy/k8s/                       # 部署所有资源
kubectl get pods -n hify                           # 确认 Pod 都 Running
kubectl scale deployment hify-backend --replicas=4 -n hify  # 按需扩容
```

验收流程（三种形态统一）：

服务起来后，走一遍完整链路

创建 Provider，测试 LLM 连通性

创建 Agent，发消息确认流式响应正常（打字机效果）

上传知识库文档，提问验证 RAG 检索注入

配置工作流，测试 CONDITION 分支路由

绑定 MCP 工具，查询订单

全部通过，Hify 从“只能在你机器上跑”变成了“任何环境都能部署”。

总结

这一讲做完，deploy 目录下多了这些东西，里面的内容我就不展开讲了，你可以去看一下源码。

复制代码

```
.
├── Dockerfile                    后端镜像
```

```
├── docker-compose.yml
├── Makefile                          含 package / docker-build 命令
├── start.sh / stop.sh                本地部署脚本（完全重写为生产模式）
├── hify-*/                           各业务模块
├── hify-web/
│   └── Dockerfile                    前端镜像
└── deploy/
    ├── application.yml.template      本地部署配置模板
    ├── env.template                  环境变量模板（本地 + Docker 通用）
    ├── docker/
    │   └── nginx.conf                前端 Nginx 配置
    └── k8s/
        ├── backend-configmap.yml
        ├── backend-secret.yml
        ├── backend-deployment.yml
        ├── backend-service.yml
        ├── frontend-deployment.yml
        └── frontend-service.vml
```

零基础手写配置文件，你做了三件事：**用自然语言描述需求、检查 Claude Code 生成结果的关键点、按顺序执行命令**。

有一个细节值得回味：健康检查接口不是我们一开始就想到的，是 Claude Code 在讨论部署形态时主动提出来的。而且在补接口时，Claude Code 没有直接生成新文件，而是先扫了项目发现接口已存在，发现格式不一致后问了你再改。生成 Dockerfile 时读了 package.json 和 vite.config.ts，生成 docker-compose 时读了 env.template 确认变量名。

这是我这一讲真正想说的：AI 不只是代码生成器，用对了是协作者。咨询模式让它帮你发现遗漏，执行前让它先读项目了解上下文，每一步它都在基于你的实际情况做决策，而不是套模板输出。

思考题

1. 当前的部署流程是手动的，每次更新代码要手动 build、push、apply。让 Claude Code 帮你写一个 GitHub Actions workflow，推送到 main 分支后自动构建镜像并部署到 K8s。描述你的需求，看它生成的 YAML 是否可以直接用，有哪些地方需要你来判断和调整。
2. K8s Secret 默认只是 base64 编码，不是加密。让 Claude Code 帮你了解更安全的方案，比如 sealed-secrets 或云厂商的密钥管理服务，对比一下各自的成本和适用场景，看

哪种适合 Hify 的目标用户。

期待你的分享！如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (5)

最新　精选



杜桀
2026-05-09 来自山东

github上的代码直接使用docker-compose部署都跑不起来，构建用的jdk基础镜像里没有mvn。 另外前后端的apiKey还是api_key不对应。在模型管理中创建模型后，创建Agent时却引用不到。

作者回复: 感谢提醒，这个我看看修复下，应该是最新代码没提交更新。这里可以让CC 辅助修复一下。提示词基本就是：把你这句话给CC，就能修复。

1



水中鱼儿
2026-06-18 来自浙江

麻烦老师把代码补一下，CLAUDE中的描述不全，应该是版本太老，只有开发前的标准，没有开发中和开发后的标准在里面，skill也都没有放进来

作者回复: 已更新，看这个仓库哈：https://github.com/socutes/robustmq-hify



xinnong
2026-05-14 来自山东

麻烦发下源码地址

作者回复: 马上放进去～

Geek_33d5cb
2026-05-03 来自湖南

老师你好，关于 本地部署：打成 tar 包，这个 dist 文件是还要手动部署的 nginx 里吗？感觉直接加载到 app.jar 目录里会不会更合适。

作者回复: dist 可以直接启动的，用java –jar 就能启动。也可以挂在再nginx后面。你这里“ app.jar 目录”指的是什么呢？有点不太理解哈，期待补充信息～

Geek_33d5cb
2026-05-03 来自湖南

帮我生成 Hify 的 docker-compose.yml。

要求：
- 包含前端和后端两个服务
- MySQL、Redis、pgvector 是外部服务，通过环境变量配置连接地址
- 后端健康检查用 /api/v1/health 接口
- 前端依赖后端健康检查通过后才启动
- 敏感配置（密码、API Key）从 .env 文件读取攒批

其中“攒批”应该是错别字~

作者回复: 感谢感谢提醒🎉，我来修改一下。